

Name: Mohammed Arshad R

Reg no:192521126

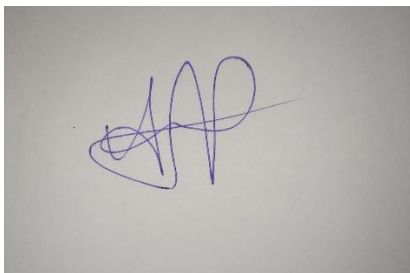
Subject: Cryptography and Network Security

Assignment: Annexure A - Analytical Problem Solving (Questions 1 & 2)

Code of Conduct:

I Mohammed Arshad R/192521126 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A handwritten signature in blue ink, appearing to be 'MAAR', is written on a light-colored surface.

CRYPTOGRAPHY & NETWORK SECURITY LABORATORY

ASSIGNMENT

QUESTION 1: RSA KEY GENERATION AND SECURITY ANALYSIS

1. Problem Overview & Initial Parameters

The RSA algorithm, named after Ron Rivest, Adi Shamir, and Leonard Adleman, is a foundational publickey cryptosystem based on the mathematical difficulty of factoring the product of two large prime numbers. In this problem, we evaluate key generation, encryption, decryption, and security analysis given small prime numbers.

Given Parameters:

- Prime 1: $p = 17$
- Prime 2: $q = 19$
- Public Exponent: $e = 5$
- Plaintext Message: $M = 7$

2. Step-by-Step Computation & Key Derivation

2.1. Calculation of Modulus n

The modulus n forms part of both the public and private keys. It is calculated as the product of primes p and q :

$$n = p * q = 17 * 19 = 323$$

2.2. Calculation of Euler's Totient Function $\phi(n)$

Euler's Totient Function calculates the number of positive integers up to n that are relatively prime to n . For two distinct primes:

$$\phi(n) = (p - 1) * (q - 1) = (17 - 1) * (19 - 1) = 16 * 18 = 288$$

2.3. Public Key Selection Validation

The public exponent e must satisfy two fundamental cryptographic requirements: $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$.

Given $e = 5$, we verify coprime condition using the Euclidean Algorithm:

$$288 = 5 * 57 + 3 \quad 5 = 3 * 1 + 2 \quad 3 = 2 * 1 + 1 \quad 2 = 1 * 2 + 0$$

Since $\gcd(5, 288) = 1$, $e = 5$ is a valid public key exponent.

2.4. Derivation of Private Key d

The private exponent d is the modular multiplicative inverse of e modulo $\phi(n)$:

$$d * e \equiv 1 \pmod{\phi(n)} \Rightarrow d * 5 \equiv 1 \pmod{288}$$

We apply the Extended Euclidean Algorithm to solve $5d - 288k = 1$:

- $1 = 3 - 2 * 1$
- Substitute $2 = 5 - 3 * 1 \Rightarrow 1 = 3 - (5 - 3 * 1) = 2 * 3 - 5 * 1$
- Substitute $3 = 288 - 5 * 57 \Rightarrow 1 = 2 * (288 - 5 * 57) - 5 * 1 = 2 * 288 - 114 * 5 - 5 = 2 * 288 - 115$

* 5 Thus, $-115 * 5 \equiv 1 \pmod{288}$. Converting to positive modulo 288:

$$d = -115 + 288 = 173$$

Verification: $(173 * 5) \pmod{288} = 865 \pmod{288} = 1$. Correct.

3. Encryption and Decryption Execution

3.1. Encryption Process

To encrypt plaintext $M = 7$ using Public Key ($e=5, n=323$):

$$C = M^e \pmod{n} = 7^5 \pmod{323}$$

Step-by-step modular exponentiation:

- $7^1 = 7 \pmod{323}$
- $7^2 = 49 \pmod{323}$
- $7^4 = (49)^2 = 2401 \pmod{323}$
- $2401 = 323 * 7 + 140 \Rightarrow 7^4 \equiv 140 \pmod{323}$
- $7^5 = (7^4 * 7) = (140 * 7) = 980 \pmod{323}$
- $980 = 323 * 3 + 11 \Rightarrow C = 11$

Resulting Ciphertext: $C = 11$

3.2. Decryption Process

To decrypt ciphertext $C = 11$ using Private Key ($d=173, n=323$):

$$M = C^d \pmod{n} = 11^{173} \pmod{323}$$

Using repeated squaring for exponent 173 (binary: 10101101):

- $11^1 = 11 \pmod{323}$
- $11^2 = 121 \pmod{323}$

- $11^4 = (121)^2 = 14641 \bmod 323 === 106 \bmod 323$
- $11^8 = (106)^2 = 11236 \bmod 323 === 252 \bmod 323$
- $11^{16} = (252)^2 = 63504 \bmod 323 === 197 \bmod 323$
- $11^{32} = (197)^2 = 38809 \bmod 323 === 5 \bmod 323$
- $11^{64} = (5)^2 = 25 \bmod 323$
- $11^{128} = (25)^2 = 625 \bmod 323 === 302 \bmod 323$ Multiplying terms for $173 = 128 + 32 + 8 + 4$

+ 1:

$$M = (11^{128} * 11^{32} * 11^8 * 11^4 * 11^1) \bmod 323 \quad M = (302 * 5 * 252 * 106 * 11) \bmod 323 = 7$$

Decrypted Message: $M = 7$ (Matches original input exactly).

4. RSA Security Analysis: Small Prime Vulnerabilities

In this problem, primes $p=17$ and $q=19$ were selected for computational demonstration. However, in realworld deployments, small prime numbers compromise security completely due to the following attacks:

4.1. Trivial Factorization Attacks

Modern integer factorization algorithms such as Trial Division, Pollard's *rho* algorithm, and the General Number Field Sieve (GNFS) can factor a small modulus $n = 323$ in microseconds. Once n is factored into p and q , an attacker reconstructs $\phi(n) = (p-1)(q-1)$ and solves $d = e^{-1} \bmod \phi(n)$, breaking all confidentiality.

4.2. Wiener's Low Private Exponent Attack

When small primes yield a small private exponent $d < (1/3) * n^{1/4}$, Wiener's attack uses continued fractions to recover private key d efficiently in polynomial time.

4.3. Modern Standard Requirements

To ensure security against state-of-the-art supercomputers and quantum algorithms:

- RSA bit length must be at least **2048 bits** (ideally 4096 bits).
- Primes p and q must be chosen uniformly at random, kept secret, and be equal in bit length.
- Padding schemes like **OAEP (Optimal Asymmetric Encryption Padding)** must be applied to prevent deterministic textbook RSA attacks.

5. Python Verification Script: RSA Problem 1

```
import math

def
run_rsa_demo():
p, q = 17, 19
e = 5      M = 7

    n = p * q      phi = (p
- 1) * (q - 1)

    # Calculate d
d = pow(e, -1, phi)

    # Encrypt
C = pow(M, e, n)

    # Decrypt      M_decrypted
= pow(C, d, n)

    print(f"Modulus n: {n}")      print(f"Totient
phi: {phi}")      print(f"Private Key d: {d}")
print(f"Encrypted Ciphertext C: {C}")
print(f"Decrypted Plaintext M: {M_decrypted}")

run_rsa_demo()
```

QUESTION 2: ELGAMAL ENCRYPTION ANALYSIS

1. Problem Overview & Initial Parameters

The ElGamal encryption scheme, designed by Taher ElGamal in 1985, is an asymmetric key algorithm based on the Diffie-Hellman key exchange principle. Its security relies on the hardness of computing Discrete Logarithms over prime finite fields $GF(p)$.

Given Parameters:

- Prime Modulus: $p = 23$
- Generator: $g = 5$
- Private Key: $x = 7$
- Random Ephemeral Nonce: $k = 3$
- Plaintext Message: $M = 10$

2. Public Key Computation

The recipient's public key y is calculated by performing modular exponentiation of generator g raised to private key x modulo p :

$$y = g^x \bmod p = 5^7 \bmod 23$$

Computation via modular exponentiation:

- $5^1 = 5 \bmod 23$
- $5^2 = 25 \bmod 23 = 2 \bmod 23$
- $5^4 = (2)^2 = 4 \bmod 23$
- $5^7 = (5^4 * 5^2 * 5^1) = (4 * 2 * 5) = 40 \bmod 23$
- $40 = 23 * 1 + 17 \Rightarrow y = 17$

Public Key Tuple: $(p=23, g=5, y=17)$

3. Encryption Phase: Generating Ciphertext Pair $(C1, C2)$

ElGamal encryption outputs a pair of ciphertexts $(C1, C2)$ using ephemeral random nonce $k=3$.

3.1. Calculation of $C1$

$C1$ represents the ephemeral public key used to transmit keying material:

$$C1 = g^k \bmod p = 5^3 \bmod 23 \quad 5^3 = 125 \bmod 23 \quad 125 = 23 * 5 + 10 \Rightarrow C1 = 10$$

3.2. Calculation of Shared Secret S and $C2$

The sender calculates shared secret $S = y^k \bmod p$ and multiplies it with plaintext message M :

$$S = y^k \bmod p = 17^3 \bmod 23$$

- $17^1 = 17 \bmod 23$
- $17^2 = (-6)^2 = 36 \bmod 23 = 13 \bmod 23$
- $17^3 = 17 * 13 = 221 \bmod 23$
- $221 = 23 * 9 + 14 \Rightarrow S = 14$

Now compute payload component $C2$:

$$C2 = (M * S) \bmod p = (10 * 14) \bmod 23 = 140 \bmod 23 \\ 140 = 23 * 6 + 2 \Rightarrow C2 = 2$$

Resulting Ciphertext Pair: $(C1 = 10, C2 = 2)$

4. Decryption Phase: Recovering Plaintext M

The recipient recovers plaintext M from ciphertext pair $(C1, C2)$ using private key $x=7$.

4.1. Shared Secret Reconstruction

The recipient reconstructs shared secret S using received $C1$:

$$S = C1^x \bmod p = 10^7 \bmod 23$$

- $10^1 = 10 \bmod 23$
- $10^2 = 100 \bmod 23 \equiv 8 \bmod 23$
- $10^4 = (8)^2 = 64 \bmod 23 \equiv 18 \bmod 23 \equiv -5 \bmod 23$
- $10^7 = (10^4 * 10^2 * 10^1) = (-5 * 8 * 10) = -400 \bmod 23 \equiv -400 \bmod 23 = 14 \bmod 23 \Rightarrow S = 14$

Reconstructed secret $S = 14$ matches encryption secret perfectly.

4.2. Modular Multiplicative Inverse of S

To isolate M , compute $S^{-1} \bmod 23$ where $(14 * S^{-1}) \equiv 1 \bmod 23$:

Using Extended Euclidean Algorithm for $14S^{-1} - 23k = 1$:

- $23 = 14 * 1 + 9$
- $14 = 9 * 1 + 5$
- $9 = 5 * 1 + 4$
- $5 = 4 * 1 + 1$

Unwinding steps:

- $1 = 5 - 4 = 5 - (9 - 5) = 2 * 5 - 9$
- $1 = 2 * (14 - 9) - 9 = 2 * 14 - 3 * 9$
- $1 = 2 * 14 - 3 * (23 - 14) = 5 * 14 - 3 * 23$

Thus, $14^{-1} \bmod 23 = 5$. (Check: $14 * 5 = 70 = 23 * 3 + 1$).

4.3. Message Decryption

$$M = (C2 * S^{-1}) \bmod p = (2 * 5) \bmod 23 = 10$$

Decrypted Plaintext: $M = 10$ (Exact recovery verified).

5. Security Analysis: Role of Randomness k in ElGamal

The introduction of per-message random nonce k (ephemeral key) provides critical cryptographic properties over deterministic systems like textbook RSA:

5.1. Probabilistic Encryption & Semantic Security

ElGamal is a **probabilistic cryptosystem**. Encrypting the exact same message M multiple times produces completely different ciphertext pairs $(C1, C2)$ because a unique random k is generated for every session. This prevents eavesdroppers from detecting message frequency patterns or conducting known-plaintext dictionary lookups.

5.2. Catastrophic Risks of Reusing Nonce k

If an implementation reuses nonce k to encrypt two different messages $M1$ and $M2$, an adversary can easily extract the plaintext ratio:

$$C2_a / C2_b = (M1 * S) / (M2 * S) == M1 / M2 \pmod{p}$$

If one message plaintext is known, all other messages encrypted with the same k are immediately compromised. Therefore, k must be generated using a cryptographically secure pseudorandom number generator (CSPRNG) and never reused.

6. Python Verification Script: ElGamal Problem 2

```
def run_elgamal_demo():
    p = 23
    g = 5
    x = 7
    k = 3
    M = 10

    # Public key calculation
    y = pow(g, x, p)

    # Encryption
    C1 = pow(g, k, p)
    S = pow(y, k, p)
    C2 = (M * S) % p

    # Decryption
    S_rec = pow(C1, x, p)
    S_inv = pow(S_rec, -1, p)
    M_decrypted = (C2 * S_inv) % p

    print(f"Public Key y: {y}")
    print(f"Ciphertext Pair (C1, C2): ({C1}, {C2})")
    print(f"Reconstructed Shared Secret S: {S_rec}")
    print(f"Decrypted Plaintext M: {M_decrypted}")

run_elgamal_demo()
```